VISVESVARAYA TECHNOLOGICAL UNIVERSITY

"JnanaSangama", Belgaum -590014, Karnataka.

LAB REPORT
on

Machine Learning (23CS6PCMAL)

Submitted by

Bhoomi Suresh Kota (1BM23CS065)

in partial fulfillment for the award of the degree of

BACHELOR OF ENGINEERING
in
COMPUTER SCIENCE AND ENGINEERING

B.M.S. COLLEGE OF ENGINEERING
(Autonomous Institution under VTU)
BENGALURU-560019
Sep-2024 to Jan-2026

B.M.S. College of Engineering,

Bull Temple Road, Bangalore 560019

(Affiliated To Visvesvaraya Technological University, Belgaum)

Department of Computer Science and Engineering

CERTIFICATE

This is to certify that the Lab work entitled "Machine Learning (23CS6PCMAL)" carried out by **Bhoomi Suresh Kota (1BM23CS065),** who is a bonafide student of **B.M.S. College of Engineering.** It is in partial fulfillment for the award of **Bachelor of Engineering in Computer Science and Engineering** of the Visvesvaraya Technological University, Belgaum. The Lab report has been approved as it satisfies the academic requirements in respect of an Machine Learning (23CS6PCMAL) work prescribed for the said degree.

Rashmi H. Assistant Professor Department of CSE, BMSCE	Dr. Kavitha Sooda Professor & HOD Department of CSE, BMSCE

Index

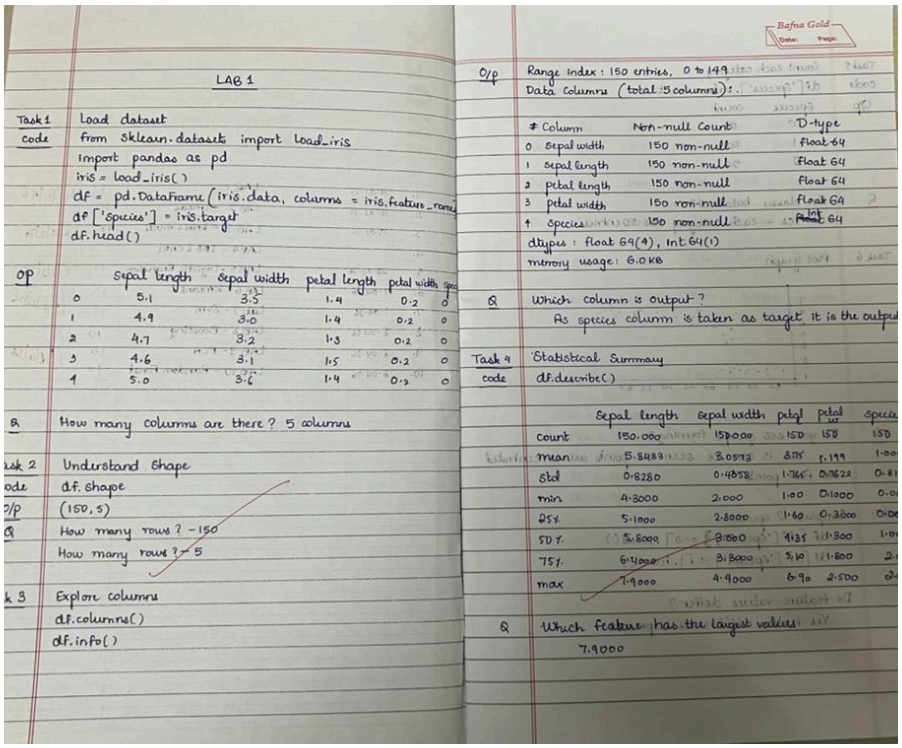
Sl. No.	Date	Experiment Title	Page No.
1	4-2-2026	Write a python program to import and export data using Pandas library functions	
2	4-2-2026	Demonstrate various data pre-processing techniques for a given dataset	
3	11-2-2026	Implement Linear and Multi-Linear Regression algorithm using appropriate dataset	
4	25-2-2026	Build Logistic Regression Model for a given dataset	
5	25-2-2026	Use an appropriate data set for building the decision tree (ID3) and apply this knowledge to classify a new sample.	
6	25-3-2026	Build KNN Classification model for a given dataset.	
7	1-4-2026	Build Support vector machine model for a given dataset	
8	1-4-2026	Implement Random forest ensemble method on a given dataset.	
9	8-4-2026	Implement Boosting ensemble method on a given dataset.	
10	8-4-2026	Build k-Means algorithm to cluster a set of data stored in a .CSV file.	
11	8-4-2026	Implement Dimensionality reduction using Principal Component Analysis (PCA) method.	

Github Link:
https://github.com/BhoomiSuresh/Machine-Learning-Lab.git

Program 1

Write a python program to import and export data using Pandas library functions

Screenshot:



LAB 1

Task 1 code
```
Load dataset
from sklearn.datasets import load_iris
import pandas as pd
iris = load_iris()
df = pd.DataFrame(iris.data, columns = iris.feature_names)
df['species'] = iris.target
df.head()
```

op

	sepal length	sepal width	petal length	petal width	species
0	5.1	3.5	1.4	0.2	0
1	4.9	3.0	1.4	0.2	0
2	4.7	3.2	1.3	0.2	0
3	4.6	3.1	1.5	0.2	0
4	5.0	3.6	1.4	0.2	0

Q How many columns are there? 5 columns

Task 2 code
```
df.shape
```
O/P (150, 5)

Q How many rows? - 150

How many rows? 5

Task 3
```
Explore columns
df.columns()
df.info()
```

O/p
```
Range index: 150 entries, 0 to 149
Data columns (total 5 columns):
```

#	Column	Non-null Count	D-type
0	sepal width	150 non-null	float 64
1	sepal length	150 non-null	float 64
2	petal length	150 non-null	float 64
3	petal width	150 non-null	float 64
4	species	150 non-null	int 64

```
dtypes: float 64(4), int 64(1)
memory usage: 6.0 KB
```

Q Which column is output?

As species column is taken as target it is the output

Task 4 code
```
df.describe()
```

	sepal length	sepal width	petal l	petal w	species
count	150.000	150.000	150	150	150
mean	5.8433	3.0573	3.75	1.199	1.00
std	0.8280	0.4358	1.765	0.7622	0.8
min	4.3000	2.000	1.00	0.1000	0.0
25%	5.1000	2.8000	1.60	0.3000	0.00
50%	5.8000	3.000	4.35	1.300	1.00
75%	6.4000	3.3000	5.10	1.800	2.
max	7.9000	4.4000	6.90	2.500	2.

Q Which feature has the largest values

7.9000

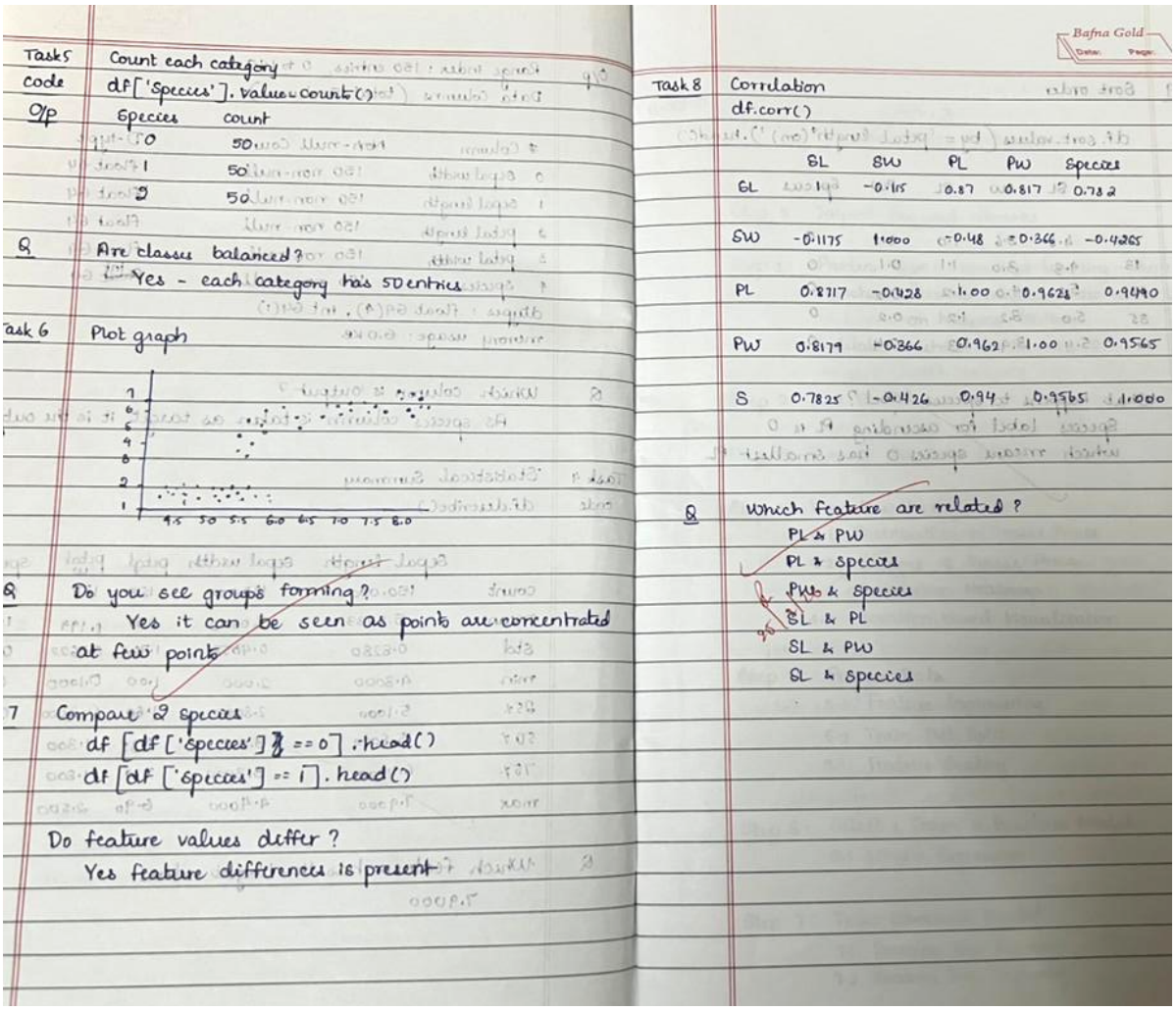
Code:

```
from sklearn.datasets import load_iris

import pandas as pd

iris=load_iris()

df =pd.DataFrame(iris.data,columns=iris.feature_names)

df['species']=iris.target

df.head()

df.shape

df.columns

df.info()

df.describe()
```

Program 2

Demonstrate various data pre-processing techniques for a given dataset.

Screenshot:



Task5 Count each category

code df['Species'].value_count()

O/p

Species	count
0	50
1	50
2	50

Q Are classes balanced?

Yes - each category has 50 entries

Task 6 Plot graph

Q Do you see groups forming?

Yes it can be seen as points are concentrated at few points

7 Compare 2 species

df [df ['species'] == 0] .head()

df [df ['species'] == 1] . head()

Do feature values differ?

Yes feature differences is present

Task 8 Correlation

df.corr()

	SL	SW	PL	PW	Species
SL		-0.115	0.87	0.817	0.782
SW	-0.1175	1.000	-0.48	-0.366	-0.4265
PL	0.8717	-0.428	1.00	0.9628	0.9490
PW	0.8179	-0.366	0.962	1.00	0.9565
S	0.7825	-0.426	0.94	0.9565	1.000

Q Which feature are related?

PL & PW

PL & species

PW & species

SL & PL

SL & PW

SL & species

Code:

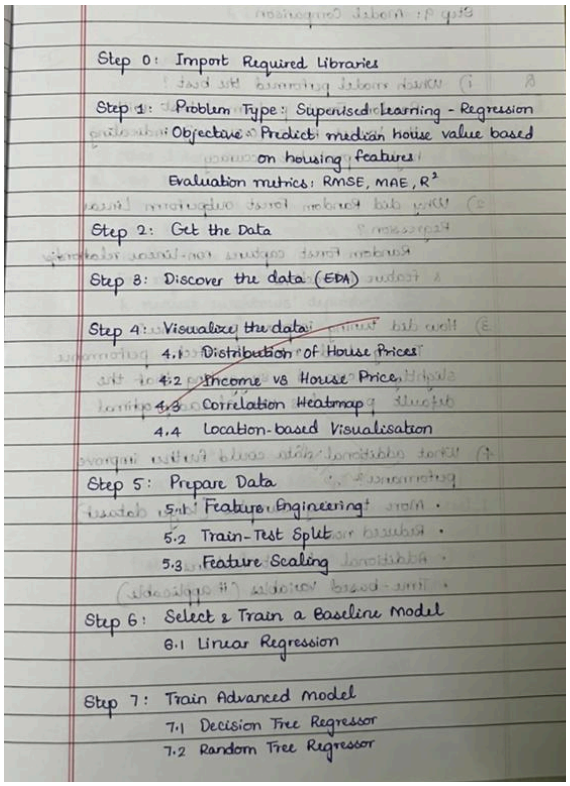
```
import pandas as pd

import numpy as np

from sklearn.preprocessing import LabelEncoder, StandardScaler

data = {

'Name': ['A', 'B', 'C', 'D', 'E'],
```

```
'Age': [20, np.nan, 22, 21, np.nan],
'Salary': [30000, 40000, 50000, np.nan, 60000],
'Department': ['HR', 'IT', 'IT', 'HR', 'Finance']
}
df = pd.DataFrame(data)
print(df)
df['Age'].fillna(df['Age'].mean(), inplace=True)
df['Salary'].fillna(df['Salary'].median(), inplace=True)
le = LabelEncoder()
df['Department'] = le.fit_transform(df['Department'])
scaler = StandardScaler()
df[['Age', 'Salary']] = scaler.fit_transform(df[['Age', 'Salary']])
df.drop_duplicates(inplace=True)
# Remove rows where Salary is too high (>2 std)
df = df[(np.abs(df['Salary']) < 2)]
X = df[['Age', 'Salary']]
y = df['Department']
print(df)
```

Program 3

Implement Linear and Multi-Linear Regression algorithm using appropriate dataset.

Screenshot:



Step 0: Import Required Libraries

Step 1: Problem Type: Supervised Learning - Regression
Objective: Predict median house value based on housing features
Evaluation metrics: RMSE, MAE, R^2

Step 2: Get the Data

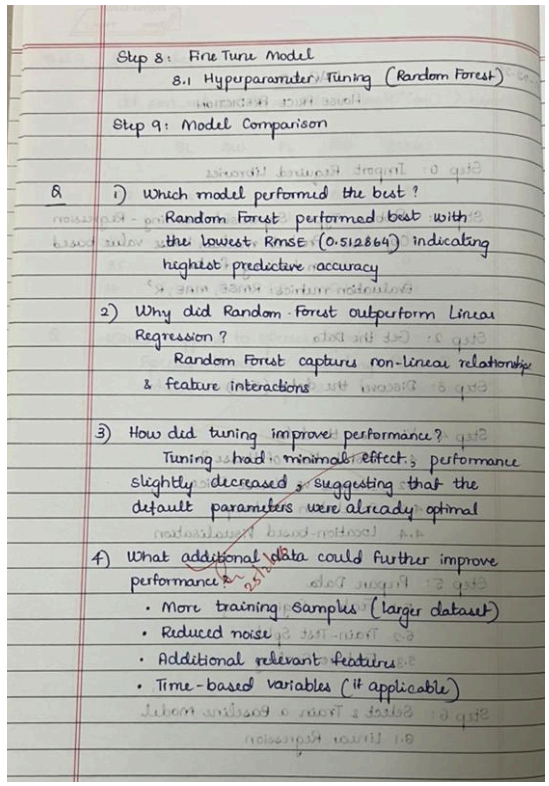
Step 3: Discover the data (EDA)

Step 4: Visualize the data
4.1 Distribution of House Prices
4.2 Income vs House Price
4.3 Correlation Heatmap
4.4 Location-based Visualisation

Step 5: Prepare Data
5.1 Feature Engineering
5.2 Train-Test Split
5.3 Feature Scaling

Step 6: Select & Train a Baseline Model
6.1 Linear Regression

Step 7: Train Advanced Model
7.1 Decision Tree Regressor
7.2 Random Tree Regressor



Step 8: Fine Tune Model
8.1 Hyperparameter Tuning (Random Forest)

Step 9: Model Comparison

8
1) Which model performed the best?
Random Forest performed best with the lowest RMSE (0.512864) indicating highest predictive accuracy

2) Why did Random Forest outperform Linear Regression?
Random Forest captures non-linear relationships & feature interactions

3) How did tuning improve performance?
Tuning had minimal effect; performance slightly decreased, suggesting that the default parameters were already optimal

4) What additional data could further improve performance?
- More training samples (larger dataset)
- Reduced noise
- Additional relevant features
- Time-based variables (if applicable)

Code:

Linear Regression

```
import numpy as np

import pandas as pd


from sklearn.model_selection import train_test_split

from sklearn.preprocessing import StandardScaler

from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score

from sklearn.linear_model import LinearRegression
```

```
data = {
    'StudyHours': [1, 2, 3, 4, 5, 6, 7, 8],
    'Attendance': [50, 55, 60, 65, 70, 75, 80, 85],
    'AssignmentMarks': [20, 25, 30, 35, 40, 45, 50, 55],
    'ExamScore': [30, 35, 40, 50, 55, 65, 75, 85]
}

df = pd.DataFrame(data)

print("Dataset:\n", df)
# 2. FEATURES AND TARGET
X = df[['StudyHours', 'Attendance', 'AssignmentMarks']]
y = df['ExamScore']
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42
)
scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
lin_reg = LinearRegression()
lin_reg.fit(X_train_scaled, y_train)
y_pred_lr = lin_reg.predict(X_test_scaled)
rmse_lr = np.sqrt(mean_squared_error(y_test, y_pred_lr))
```

```
mae_lr = mean_absolute_error(y_test, y_pred_lr)
r2_lr = r2_score(y_test, y_pred_lr)
print("RMSE:", rmse_lr)
print("MAE:", mae_lr)
print("R2 Score:", r2_lr)
```

Multiple Linear Regression

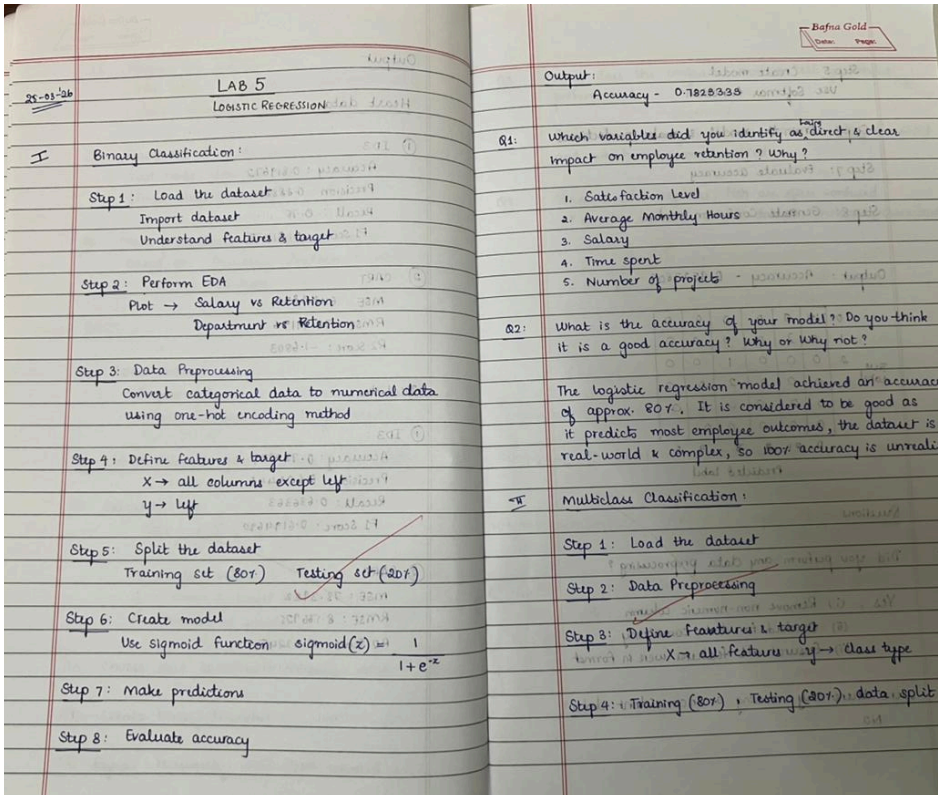
```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error
# 1. Dataset
data = {
    'StudyHours': [1, 2, 3, 4, 5, 6, 7, 8],
    'Attendance': [50, 55, 60, 65, 70, 75, 80, 85],
    'AssignmentMarks': [20, 25, 30, 35, 40, 45, 50, 55],
    'ExamScore': [30, 35, 40, 50, 55, 65, 75, 85]
}

df = pd.DataFrame(data)
# 2. Features and Target
X = df[['StudyHours', 'Attendance', 'AssignmentMarks']]
y = df['ExamScore']
# 3. Train-Test Split
X_train, X_test, y_train, y_test = train_test_split(
```

```
    X, y, test_size=0.25, random_state=42
)
# 4. Train Multilinear Model
model = LinearRegression()
model.fit(X_train, y_train)
# 5. Predictions on Test Data
predictions = model.predict(X_test)


print("Predicted Scores:", predictions)
print("MSE:", mean_squared_error(y_test, predictions))
# 6. Predict for New Student (FIXED - No Warning)
new_student = pd.DataFrame(
    [[6, 78, 48]],
    columns=['StudyHours', 'Attendance', 'AssignmentMarks']
)
predicted_score = model.predict(new_student)
print("Predicted Exam Score for new student:", predicted_score)
```

Program 4

Build Logistic Regression Model for a given dataset

Screenshot:



25-03-'26

LAB 5

LOGISTIC REGRESSION

I Binary Classification:

Step 1: Load the dataset
Import dataset
Understand features & target

Step 2: Perform EDA
Plot → Salary vs Retention
Department vs Retention

Step 3: Data Preprocessing
Convert categorical data to numerical data using one-hot encoding method

Step 4: Define features & target
X → all columns except left
y → left

Step 5: Split the dataset
Training set (80%) Testing set (20%)

Step 6: Create model
Use sigmoid function sigmoid(z) = $\frac{1}{1+e^{-z}}$

Step 7: Make predictions

Step 8: Evaluate accuracy

Output:
Accuracy - 0.7828338

Q1: Which variables did you identify as having direct & clear impact on employee retention? Why?

1. Satisfaction Level
2. Average Monthly Hours
3. Salary
4. Time spent
5. Number of projects

Q2: What is the accuracy of your model? Do you think it is a good accuracy? Why or Why not?

The logistic regression model achieved an accurac of approx. 80%. It is considered to be good as it predicts most employee outcomes, the dataset is real-world & complex, so 100% accuracy is unreali

II Multiclass Classification:

Step 1: Load the dataset

Step 2: Data Preprocessing

Step 3: Define features & target
X → all features y → class type

Step 4: Training (80%), Testing (20%) data split

Code:

```
# Logistic Regression - Binary Classification (HR Dataset)


import pandas as pd

import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split

from sklearn.linear_model import LogisticRegression

from sklearn.metrics import accuracy_score, confusion_matrix, ConfusionMatrixDisplay
```

```
# Load dataset
df = pd.read_csv("HR_comma_sep.csv")

# ------------------------
# EDA (Exploratory Analysis)
# ------------------------

# Check data
print(df.head())

# Impact of salary on retention
pd.crosstab(df.salary, df.left).plot(kind='bar')
plt.title("Employee Retention vs Salary")
plt.xlabel("Salary")
plt.ylabel("Count")
plt.show()

# Impact of department on retention
pd.crosstab(df.Department, df.left).plot(kind='bar')
plt.title("Employee Retention vs Department")
plt.xlabel("Department")
plt.ylabel("Count")
plt.show()
```

```
# ------------------------
# Feature Selection
# ------------------------

# Convert categorical variables
df = pd.get_dummies(df, columns=['salary', 'Department'], drop_first=True)

# Features & Target
X = df.drop('left', axis=1)
y = df['left']

# ------------------------
# Train-Test Split
# ------------------------
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# ------------------------
# Model Training
# ------------------------
model = LogisticRegression(max_iter=1000)
model.fit(X_train, y_train)

# ------------------------
# Prediction
```

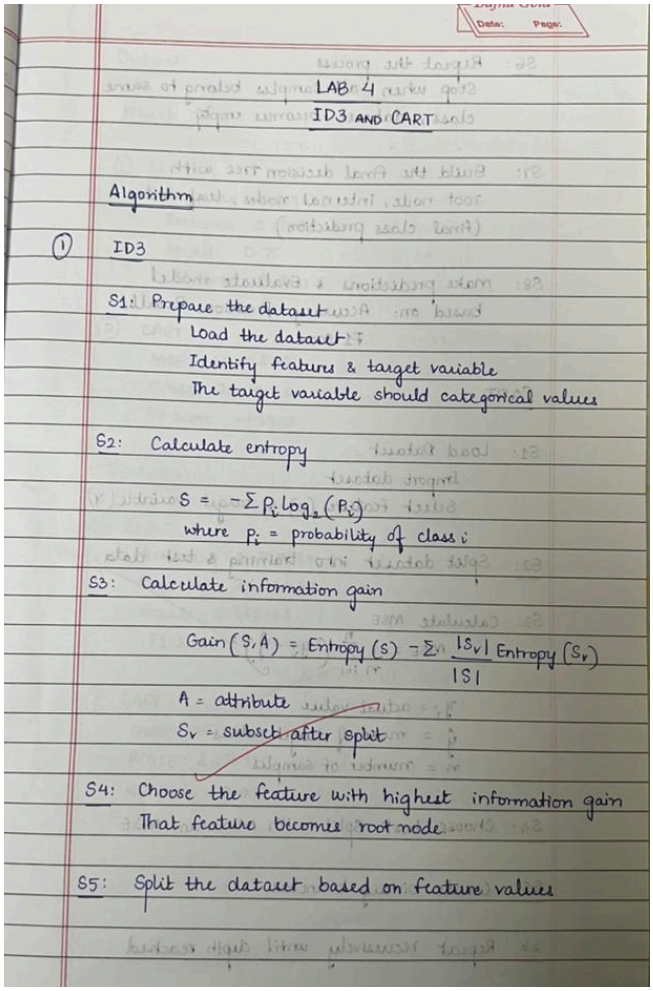
```
# ------------------------
y_pred = model.predict(X_test)

# ------------------------
# Accuracy
# ------------------------
accuracy = accuracy_score(y_test, y_pred)
print("Accuracy:", accuracy)

# ------------------------
# Confusion Matrix
# ------------------------
cm = confusion_matrix(y_test, y_pred)
disp = ConfusionMatrixDisplay(confusion_matrix=cm)
disp.plot()
plt.show()

# Logistic Regression - Multiclass Classification (Zoo Dataset)

import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix, ConfusionMatrixDisplay
import matplotlib.pyplot as plt
```

```
# Load dataset
df = pd.read_csv("zoo-data.csv")


# Drop animal name (not useful)
df = df.drop("animal_name", axis=1)
# Features & Target
X = df.drop("class_type", axis=1)
y = df["class_type"]
# Train-Test Split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
# Model (Multiclass)
model = LogisticRegression(max_iter=1000, multi_class='multinomial')
model.fit(X_train, y_train)
# Predictions
y_pred = model.predict(X_test)
# Accuracy
accuracy = accuracy_score(y_test, y_pred)
print("Accuracy:", accuracy)
# Confusion Matrix
cm = confusion_matrix(y_test, y_pred)
disp = ConfusionMatrixDisplay(confusion_matrix=cm)
disp.plot()
```

Program 5

Use an appropriate data set for building the decision tree (ID3) and apply this knowledge to classify a new sample.

Screenshot:



LAB 4

ID3 AND CART

Algorithm

① ID3

S1: Prepare the dataset
Load the dataset
Identify features & target variable
The target variable should categorical values

S2: Calculate entropy

$$S = -\sum p_i \log_2(p_i)$$

where p_i = probability of class i

S3: Calculate information gain

$$Gain(S, A) = Entropy(S) - \sum \frac{|S_v|}{|S|} Entropy(S_v)$$

A = attribute
S_v = subset after split

S4: Choose the feature with highest information gain
That feature becomes root node

S5: Split the dataset based on feature values

Code:

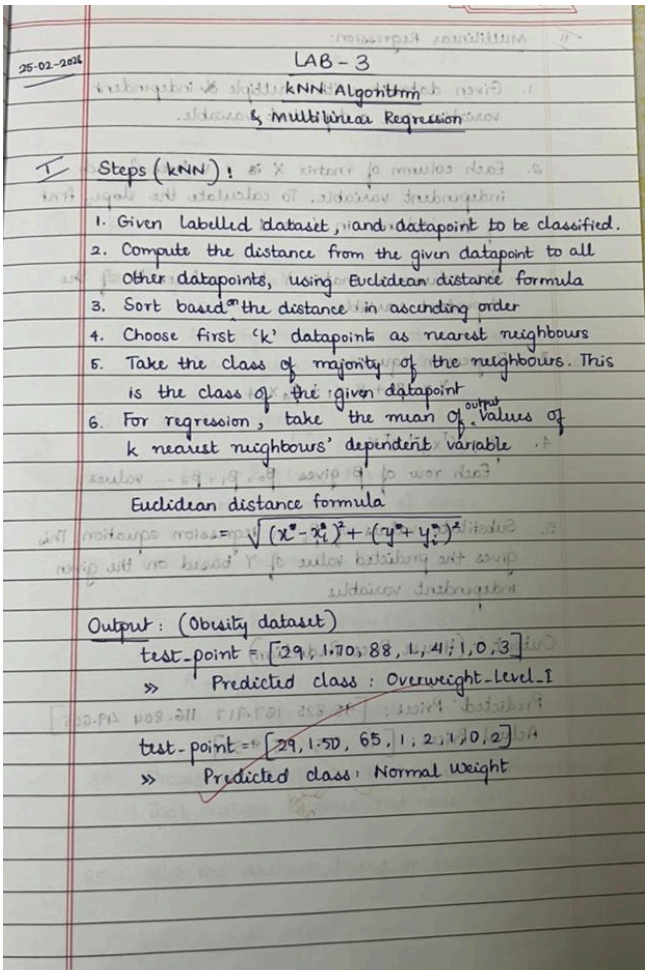
```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score

print("------ Diabetes Dataset ------")

df = pd.read_csv("diabetes.csv")

X = df.drop("Outcome", axis=1)
```

```
y = df["Outcome"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42
)

# ID3 (Entropy)
clf = DecisionTreeClassifier(criterion="entropy")
clf.fit(X_train, y_train)

pred = clf.predict(X_test)

print("\nID3 Classification Metrics:")
print("Accuracy :", accuracy_score(y_test, pred))
print("Precision:", precision_score(y_test, pred))
print("Recall   :", recall_score(y_test, pred))
print("F1 Score :", f1_score(y_test, pred))
print("\n------ Heart Dataset ------")
df = pd.read_csv("heart.csv")
X = df.drop("target", axis=1)
y = df["target"]
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42
)
# ID3 (Entropy)
clf = DecisionTreeClassifier(criterion="entropy")
clf.fit(X_train, y_train)
pred = clf.predict(X_test)
print("\nID3 Classification Metrics:")
print("Accuracy :", accuracy_score(y_test, pred))
print("Precision:", precision_score(y_test, pred))
print("Recall   :", recall_score(y_test, pred))
print("F1 Score :", f1_score(y_test, pred))
```

Program 6

Build KNN Classification model for a given dataset.

Screenshot:



25-02-2024

LAB - 3

kNN Algorithm

& Multilinear Regression

I Steps (kNN) :

1. Given labelled dataset, and datapoint to be classified.
2. Compute the distance from the given datapoint to all other datapoints, using Euclidean distance formula
3. Sort based on the distance in ascending order
4. Choose first 'k' datapoints as nearest neighbours
5. Take the class of majority of the neighbours. This is the class of the given datapoint
6. For regression, take the mean of output values of k nearest neighbours' dependent variable

Euclidean distance formula

$$= \sqrt{(x^2 - x_i^2)^2 + (y^2 + y_i^2)^2}$$

Output : (Obesity dataset)

test_point = [29, 1.70, 88, 1, 4, 1, 0, 3]

» Predicted class : Overweight_Level_I

test_point = [29, 1.50, 65, 1, 2, 1, 0, 2]

» Predicted class : Normal Weight

Code:

```
import numpy as np

import pandas as pd

from sklearn.model_selection import train_test_split

from sklearn.preprocessing import StandardScaler

from sklearn.neighbors import KNeighborsRegressor

from sklearn.metrics import mean_squared_error
```

```
data = {
    'Height': [170, 160, 180, 175, 155, 190, 165, 185],
    'Weight': [80, 75, 70, 110, 90, 85, 60, 120],
    'ObesityScore': [40, 50, 20, 80, 70, 30, 15, 90] # Target to predict
}
df = pd.DataFrame(data)

# 2. Features and Target
X = df[['Height', 'Weight']]
y = df['ObesityScore']

# 3. Split data
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, random_state=42)

# 4. Feature Scaling (Crucial for KNN)
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# 5. KNN Regression Model
knn_reg = KNeighborsRegressor(n_neighbors=3)
knn_reg.fit(X_train_scaled, y_train)

# 6. Predict and Evaluate
```

```
predictions = knn_reg.predict(X_test_scaled)

print("Predicted Scores:", predictions)

print("MSE:", mean_squared_error(y_test, predictions))


# 7. Convert Regression to "Obese or Not" (e.g., threshold > 50)

threshold = 50

binary_prediction = ["Obese" if p > threshold else "Not Obese" for p in predictions]

print("Classification:", binary_prediction)
```

Program 7

Build Support vector machine model for a given dataset

Screenshot:

1.04.26

LAB 7

SVM

Algorithm :

1. Input training dataset with class labels
2. Plot the data point in an n-d space
3. Choose a kernel function (Linear, Polynomial, RBF, Sigmoid)
4. Find the hyperplane
5. Maximize the margin between the hyperplane & nearest data points
6. Identify support vectors
7. Construct the optimal hyperplane
8. Use the model to classify

Output :

Confusion Matrix :

[[109 7]
[34 59]]

Classification Report :

	precision	recall	f1-score	support
0	0.76	0.94	0.84	116
1	0.89	0.63	0.74	93
accuracy			0.80	209
macro avg	0.83	0.79	0.79	209
weighted avg	0.82	0.80	0.80	209

Code:

```
# Titanic Survival Prediction using SVM (All-in-One)

import pandas as pd

from sklearn.model_selection import train_test_split

from sklearn.preprocessing import StandardScaler

from sklearn.svm import SVC
```

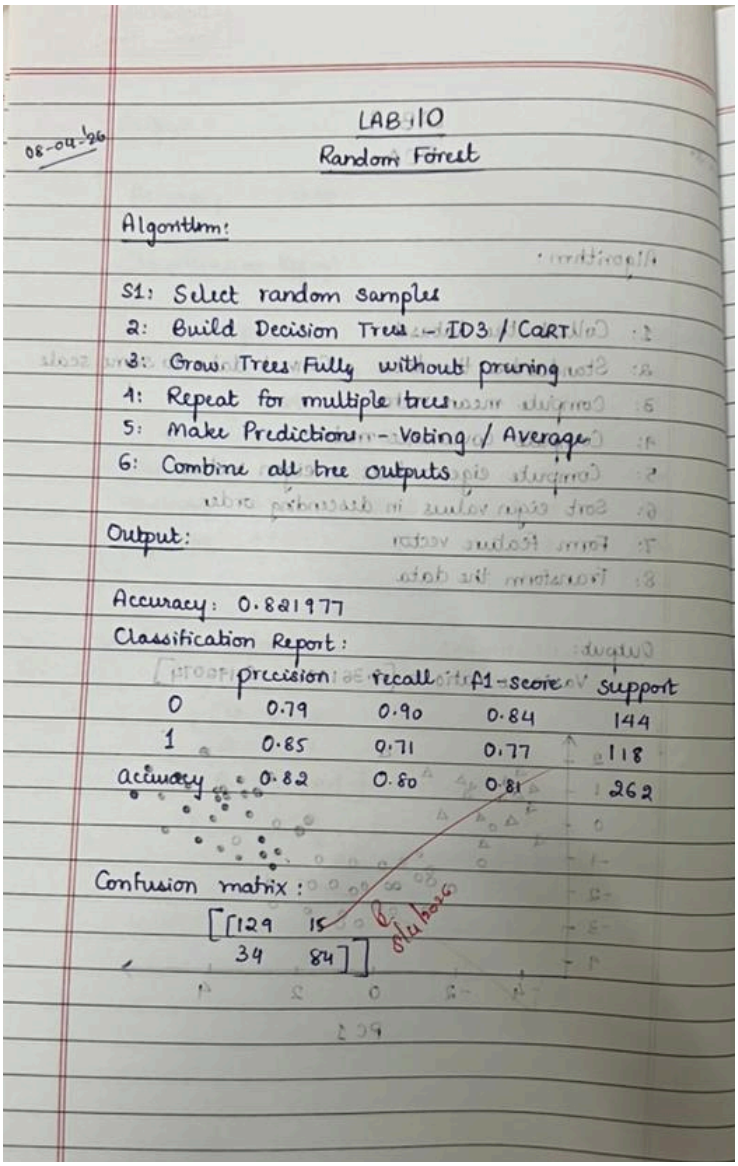
```
from sklearn.metrics import confusion_matrix, classification_report


# STEP 1: Load dataset

url = "https://raw.githubusercontent.com/rolandmueller/titanic/main/titanic3.csv"

df = pd.read_csv(url)

# STEP 2: Select relevant columns

df = df[['pclass','sex','age','fare','survived']]

# STEP 3: Handle missing values

df = df.dropna()

# STEP 4: Convert categorical to numeric

df['sex'] = df['sex'].map({'male': 0, 'female': 1})

# STEP 5: Define features and target

X = df[['pclass','sex','age','fare']]

y = df['survived']

# STEP 6: Train-test split

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)

# STEP 7: Feature scaling (VERY IMPORTANT for SVM)

scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)

X_test = scaler.transform(X_test)

# STEP 8: Train SVM model

model = SVC(kernel='rbf')

model.fit(X_train, y_train)

# STEP 9: Predictions
```

```
y_pred = model.predict(X_test)

# STEP 10: Evaluation

print("Accuracy:", model.score(X_test, y_test))

print("\nConfusion Matrix:\n", confusion_matrix(y_test, y_pred))

print("\nClassification Report:\n", classification_report(y_test, y_pred))
```

Program 8

Implement Random forest ensemble method on a given dataset.

Screenshot:



08-04-26

LAB 10

Random Forest

Algorithm:

S1: Select random samples

2: Build Decision Trees - ID3 / CART

3: Grow Trees Fully without pruning

4: Repeat for multiple trees

5: Make Predictions - Voting / Average

6: Combine all tree outputs

Output:

Accuracy: 0.821977

Classification Report:

	precision	recall	f1-score	support
0	0.79	0.90	0.84	144
1	0.85	0.71	0.77	118
accuracy	0.82	0.80	0.81	262

Confusion matrix:

[[129 15
34 84]]

8/4/26

Code:

```
import pandas as pd

import numpy as np


from sklearn.model_selection import train_test_split

from sklearn.ensemble import RandomForestClassifier

from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
```

```
df = pd.read_csv("https://raw.githubusercontent.com/rolandmueller/titanic/main/titanic3.csv")

print("First 5 rows:")
print(df.head())
df = df[["pclass", "sex", "age", "fare", "survived"]]
df["sex"] = df["sex"].map({"male": 0, "female": 1})
df["age"] = df["age"].fillna(df["age"].mean())
df["fare"] = df["fare"].fillna(df["fare"].mean())
print("\nAfter preprocessing:")
print(df.head())
X = df.drop("survived", axis=1)
y = df["survived"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
model = RandomForestClassifier(
    n_estimators=100,
    max_depth=None,
    random_state=42
)

model.fit(X_train, y_train)
```

```
y_pred = model.predict(X_test)

print("\nAccuracy:", accuracy_score(y_test, y_pred))

print("\nClassification Report:")
print(classification_report(y_test, y_pred))

print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred))

feature_importance = pd.Series(
    model.feature_importances_,
    index=X.columns
)

print("\nFeature Importance:")
print(feature_importance.sort_values(ascending=False))
```

Program 9

Implement Boosting ensemble method on a given dataset.

Screenshot:

8-04-26

LAB 8

Boosting

Ensemble

Algorithm:

1. Initialize Weights
   Assign equal weights to all training data points
2. Train first weak model
   Train a weak learner
3. Evaluate errors
   Check ~~the~~ which data points are misclassified.
4. Update weights
   Increase weights of incorrectly classified points and decrease weights of correctly classified points
5. Train next weak model
6. Repeat iteratively
7. Assign model weights & combine models

Code:

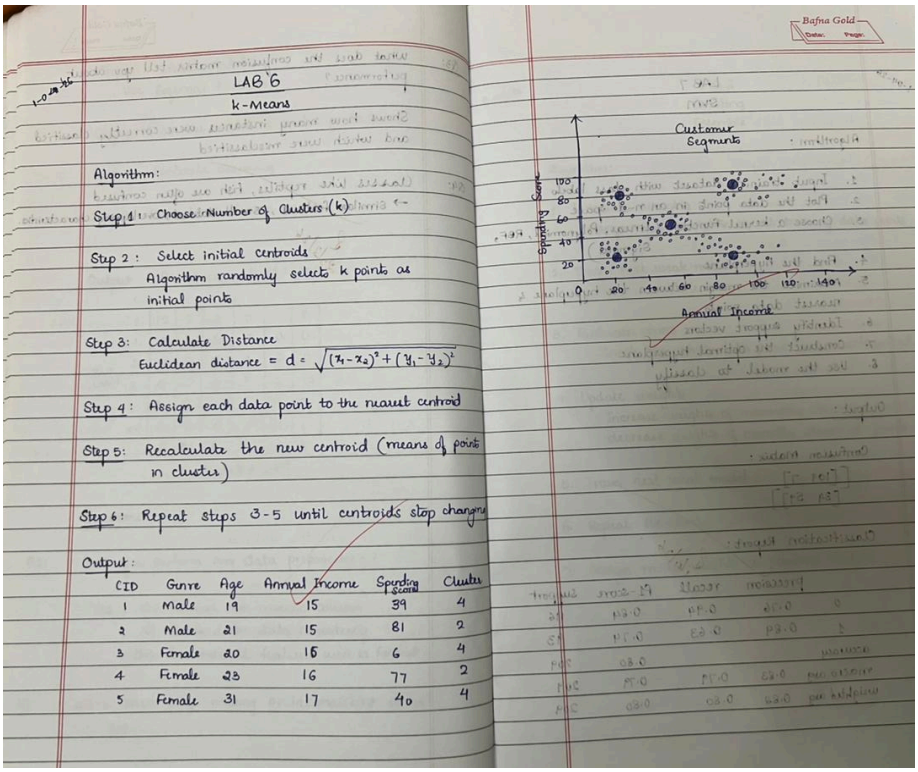
```
from sklearn.datasets import load_iris

from sklearn.model_selection import train_test_split

from sklearn.ensemble import AdaBoostClassifier

from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

import matplotlib.pyplot as plt

import pandas as pd

iris = load_iris()

X = iris.data

y = iris.target

feature_names = iris.feature_names

X_train, X_test, y_train, y_test = train_test_split(

    X, y, test_size=0.2, random_state=42)
```

```
model = AdaBoostClassifier(
    n_estimators=50,
    learning_rate=1.0,
    random_state=42
)
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
acc = accuracy_score(y_test, y_pred)
print("Accuracy:", acc)
print("\nClassification Report:")
print(classification_report(y_test, y_pred))
cm = confusion_matrix(y_test, y_pred)
print("\nConfusion Matrix:")
print(cm)
importance = pd.Series(model.feature_importances_, index=feature_names)
print("\nFeature Importance:")
print(importance)
importance.plot(kind='bar')
plt.title("AdaBoost Feature Importance")
plt.ylabel("Importance Score")
```

Program 10

Build k-Means algorithm to cluster a set of data stored in a .CSV file.

Screenshot:



LAB 6

k-Means

Algorithm:

Step 1: Choose Number of Clusters (k)

Step 2: Select initial centroids
Algorithm randomly selects k points as initial points

Step 3: Calculate Distance
Euclidean distance = $d = \sqrt{(x_1-x_2)^2+(y_1-y_2)^2}$

Step 4: Assign each data point to the nearest centroid

Step 5: Recalculate the new centroid (means of points in cluster)

Step 6: Repeat steps 3-5 until centroids stop changing

Output:

CID	Gnre	Age	Annual Income	Spending Score	Cluster
1	Male	19	15	39	4
2	Male	21	15	81	2
3	Female	20	16	6	4
4	Female	23	16	77	2
5	Female	31	17	40	4

Code:

```
!pip install kagglehub

import kagglehub

import pandas as pd

import numpy as np

import matplotlib.pyplot as plt

from sklearn.cluster import KMeans

path = kagglehub.dataset_download("abdallahwagih/mall-customers-segmentation")

print("Path to dataset files:", path)

import os

print(os.listdir(path))

df = pd.read_csv(path + "/Mall_Customers.csv")

df.head()
```

```
df.info()
df.describe()
X = df[['Annual Income (k$)', 'Spending Score (1-100)']]
wcss = []
for i in range(1, 11):
    kmeans = KMeans(n_clusters=i, random_state=42)
    kmeans.fit(X)
    wcss.append(kmeans.inertia_)
plt.plot(range(1,11), wcss)
plt.title('Elbow Method')
plt.xlabel('Number of clusters')
plt.ylabel('WCSS')
plt.show()
kmeans = KMeans(n_clusters=5, random_state=42)
y_kmeans = kmeans.fit_predict(X)
df['Cluster'] = y_kmeans
df.head()
plt.scatter(X.iloc[:,0], X.iloc[:,1], c=y_kmeans, cmap='rainbow')
plt.scatter(kmeans.cluster_centers_[:,0], kmeans.cluster_centers_[:,1], s=200, c='black')
plt.show()
```

Program 11

Implement Dimensionality reduction using Principal Component Analysis (PCA) method.

Screenshot:

8-04-26

LAB 9

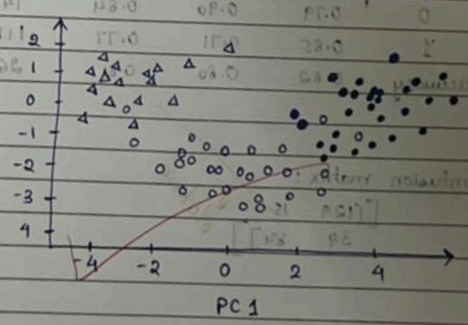
PCA

Algorithm:

1: Collect the dataset
2: Standardize the data → Convert data to same scale
3: Compute mean vector
4: Compute covariance matrix
5: Compute eigenvalues & eigen vectors
6: Sort eigen values in descending order
7: Form feature vector
8: Transform the data

Output:

Variance ratio = [0.361988 0.190074]



Code:

```
!pip install kagglehub

import kagglehub

import pandas as pd

import numpy as np

import matplotlib.pyplot as plt
```

```
path = kagglehub.dataset_download("tawfikelmetwally/wine-dataset")


print("Path to dataset files:", path)

import os

print(os.listdir(path))

df = pd.read_csv(path + "/Wine dataset.csv")

df.head()

X = df.drop("class", axis=1)   # features

y = df["class"]                # target

from sklearn.preprocessing import StandardScaler


scaler = StandardScaler()

X_scaled = scaler.fit_transform(X)

from sklearn.decomposition import PCA


pca = PCA(n_components=2)

X_pca = pca.fit_transform(X_scaled)

print("Explained Variance Ratio:", pca.explained_variance_ratio_)

import matplotlib.pyplot as plt


plt.scatter(X_pca[:,0], X_pca[:,1], c=y)

plt.xlabel("PC1")

plt.ylabel("PC2")

plt.title("PCA on Wine Dataset")
```

```
plt.show()

df.columns
```